UTEC
Universidad Tecnológica

Patrones de diseño

Los patrones de diseño son soluciones generales y reutilizables para problemas comunes que surgen durante el diseño de software. Se trata de un enfoque estructurado para resolver problemas de diseño, en lugar de escribir código desde cero cada vez que surge un problema. Un patrón de diseño no es un fragmento de código listo para usar, sino más bien una plantilla que puede aplicarse a una situación particular para resolver un problema de diseño de software.

¿Para qué sirven los Patrones de Diseño?

Los patrones de diseño sirven para:

- **Reutilizar soluciones probadas:** Ayudan a reutilizar soluciones que han sido probadas y demostradas eficaces en múltiples contextos.
- **Facilitar la comunicación:** Proporcionan un vocabulario común entre los desarrolladores. Decir "usemos un Singleton" es mucho más claro y eficiente que explicar todos los detalles del patrón cada vez.
- **Mejorar la mantenibilidad:** Fomentan la creación de código que es más fácil de entender y mantener, lo que reduce los errores y los costos de mantenimiento a largo plazo.
- **Promover buenas prácticas:** Establecen buenas prácticas de diseño que resultan en software más robusto y flexible.

¿Cómo se clasifican los Patrones de Diseño?

Los patrones de diseño se clasifican en tres categorías principales según su propósito:

1. **Patrones de Diseño Creacionales:**
    - Estos patrones se enfocan en la creación de objetos. El objetivo es abstraer el proceso de instanciación, haciendo que el sistema sea más independiente de cómo se crean, componen y representan los objetos.
    - Ejemplos: Singleton, Factory Method, Abstract Factory, Builder, Prototype.
2. **Patrones de Diseño Estructurales:**
    - Estos patrones se centran en la composición de clases y objetos. Ayudan a garantizar que si una parte del sistema cambia, el sistema completo no se vea afectado.
    - Ejemplos: Adapter, Composite, Proxy, Decorator, Facade, Bridge, Flyweight.
3. **Patrones de Diseño de Comportamiento:**

- Estos patrones están relacionados con la interacción y responsabilidad entre los objetos. Se centran en cómo los objetos colaboran y se comunican entre sí.
- Ejemplos: Strategy, Observer, Command, State, Chain of Responsibility, Mediator, Visitor, Iterator, Memento, Template Method, Interpreter.